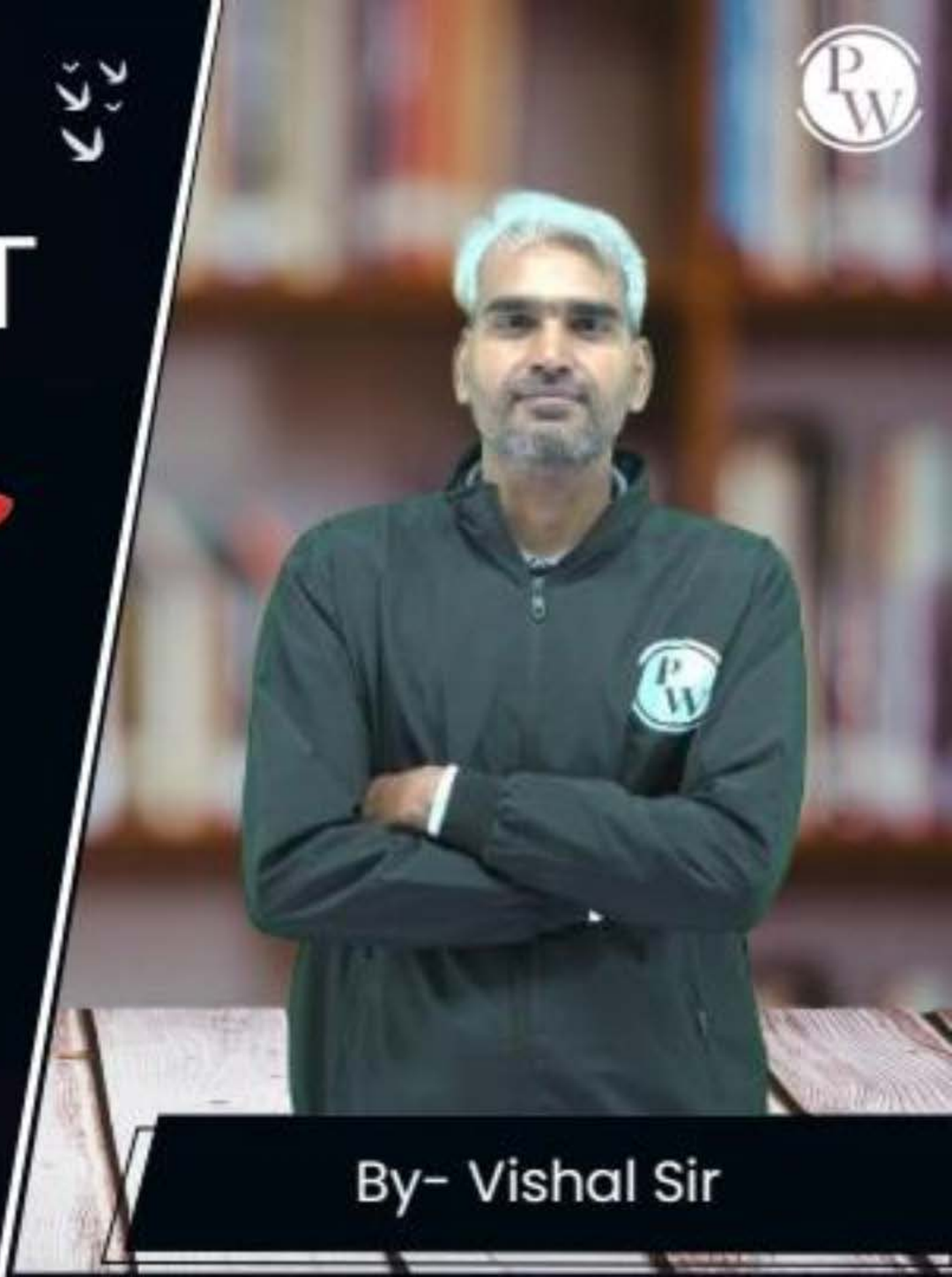
Computer Science & IT



Database Management System

Transaction & concurrency control

Lecture No. 06



By- Vishal Sir

Recap of Previous Lecture



- Topic: Classification of schedule
- Topic: Irrecoverable schedule
- Topic: Recoverable schedule
- Topic: Cascading rollback problem
- Topic: Cascadeless schedule
- Topic: Strict recoverable schedule

Slide

Topics to be Covered



- Topic: Classification based on serializability
- Topic: Conflicting and non-conflicting operations
- Topic: Conflict serializable schedule
- Topic: Precedence graph
- Topic: Topological order

Slide



Topic : Classification of schedule

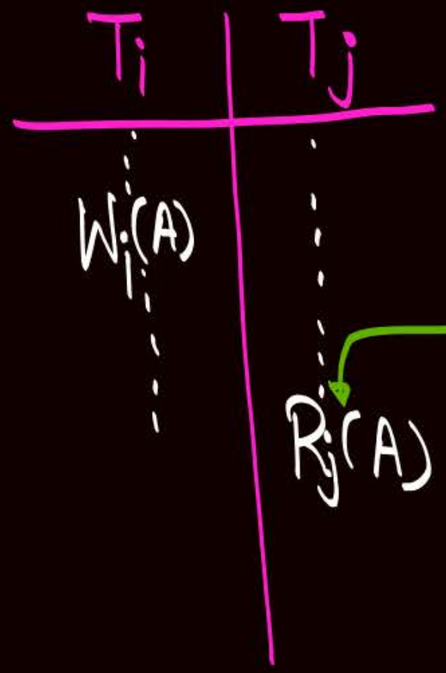
Classification based on Recoverability

1. Irrecoverable schedule
2. Recoverable schedule
3. Cascadeless rollback recoverable schedule
4. Strict recoverable schedule

Classification based on Serializability

1. Conflict serializable schedule
2. View serializable schedule

Note :- If transaction T_j reads the value written by an uncommitted transaction T_i, then transaction T_j is dependent on transaction T_i.



T_j reads the value of dataitem 'A' written by T_i so T_j is dependent on T_i

If transaction T_i rollback for some reason then we also need to rollback transactions dependent on transaction T_i {i.e, T_j in our eg.}

We can rollback a transaction only if that transaction has not executed its Commit operation

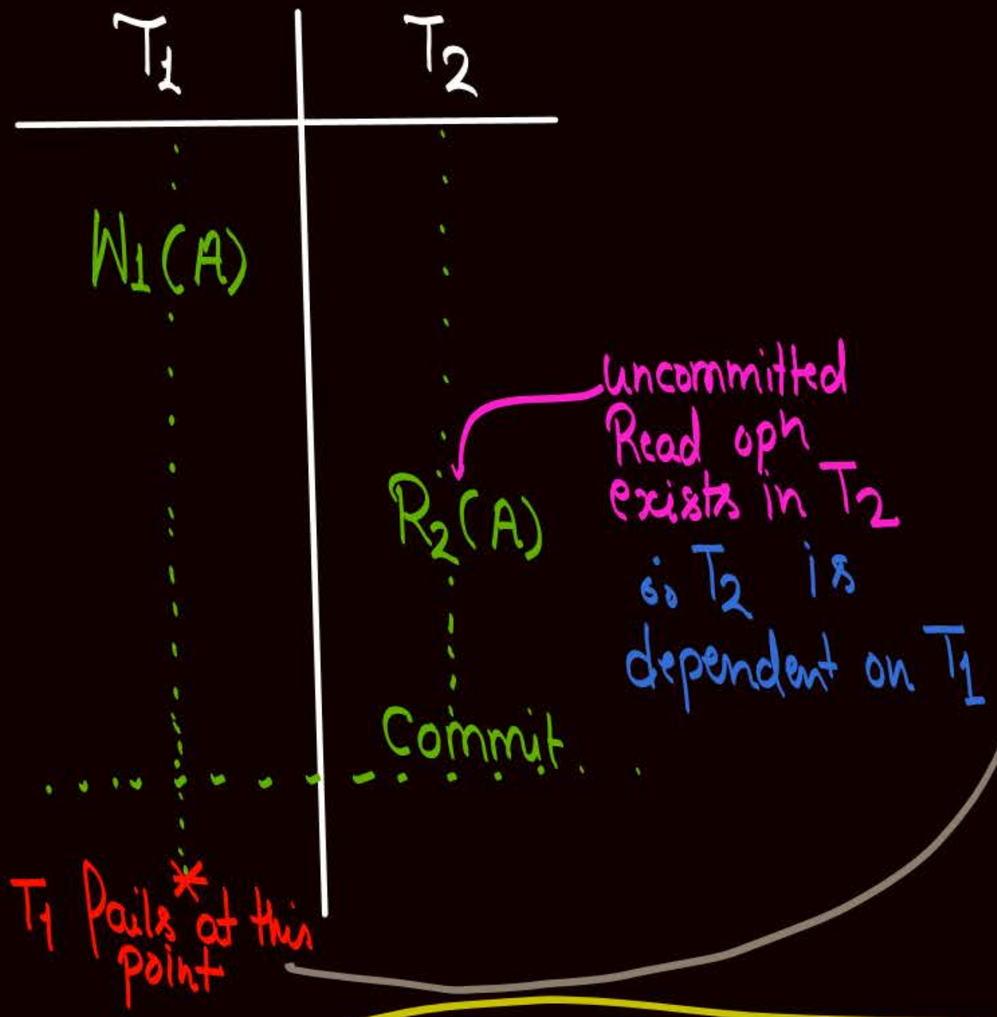
Rollback of a committed transaction is not allowed.



Topic : Irrecoverable schedule

- It is not possible to rollback a committed transaction.
- If there arise a situation when we are required to rollback a committed transaction, then it will not be possible to rollback that transaction. And hence we will not be able to recover from that failure, such schedules are called irrecoverable schedule

Irrecoverable Schedule



T1 failed, ∴ We need to rollback transaction T1.

Because of rollback of T1 We need to rollback all the transactions that are dependent on T1 { in our example, we need to rollback T2 }

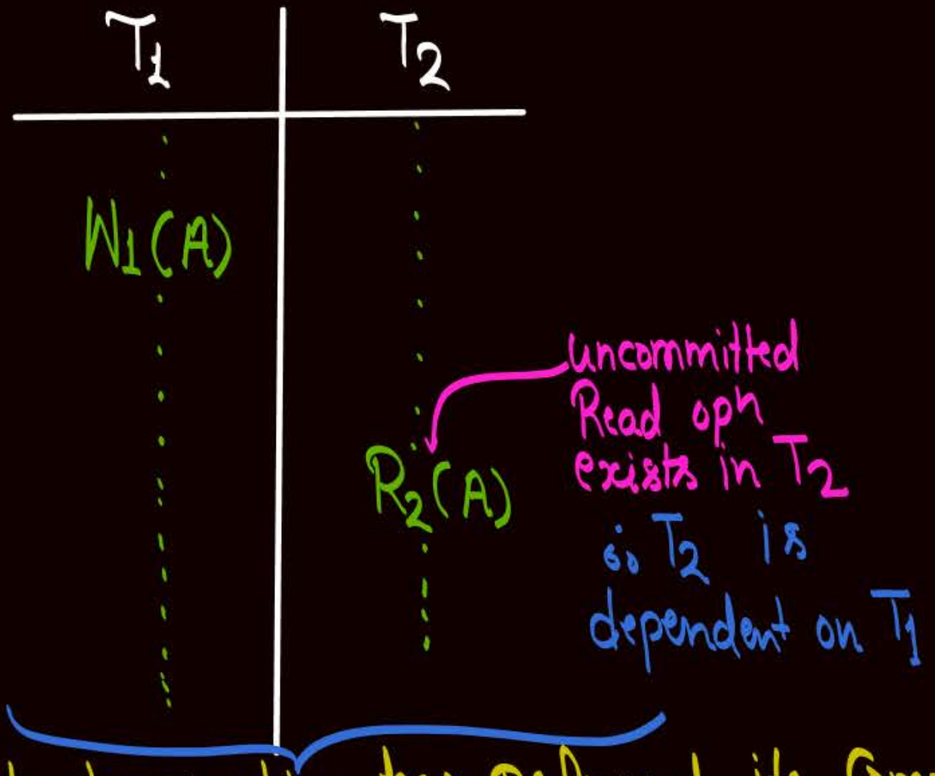
But T2 has committed ∴ We can not rollback T2

Hence we can not recover from this failure.

Therefore

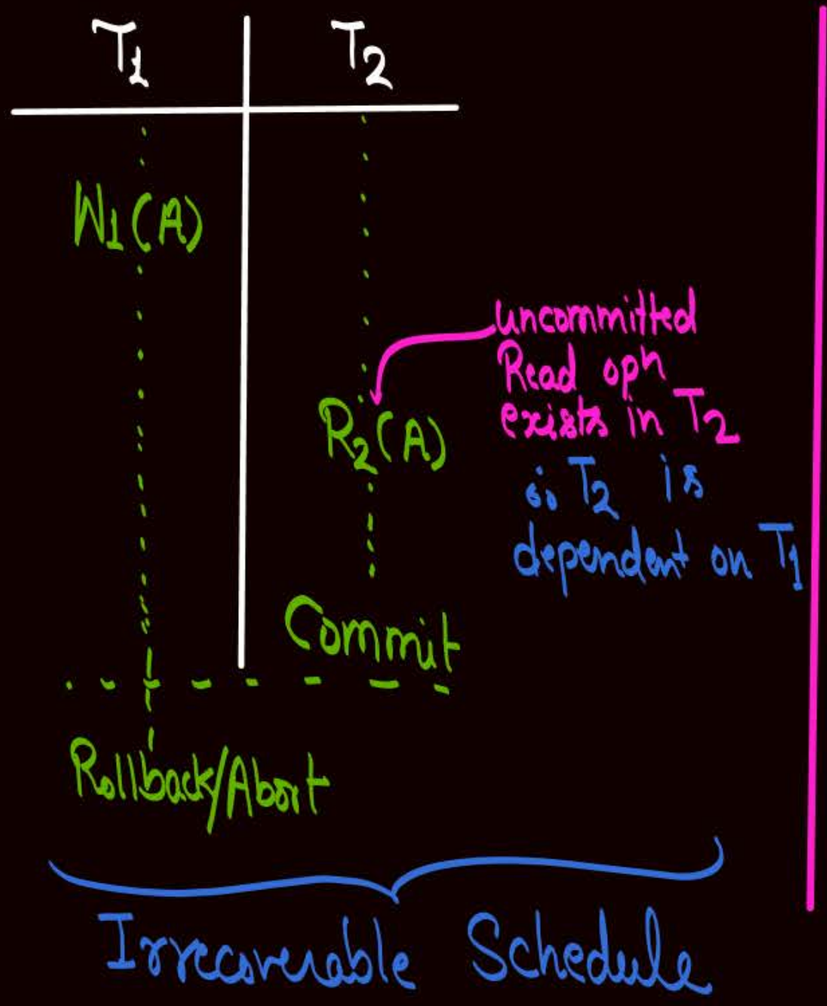
Given schedule is an irrecoverable schedule

Irrecoverable Schedule

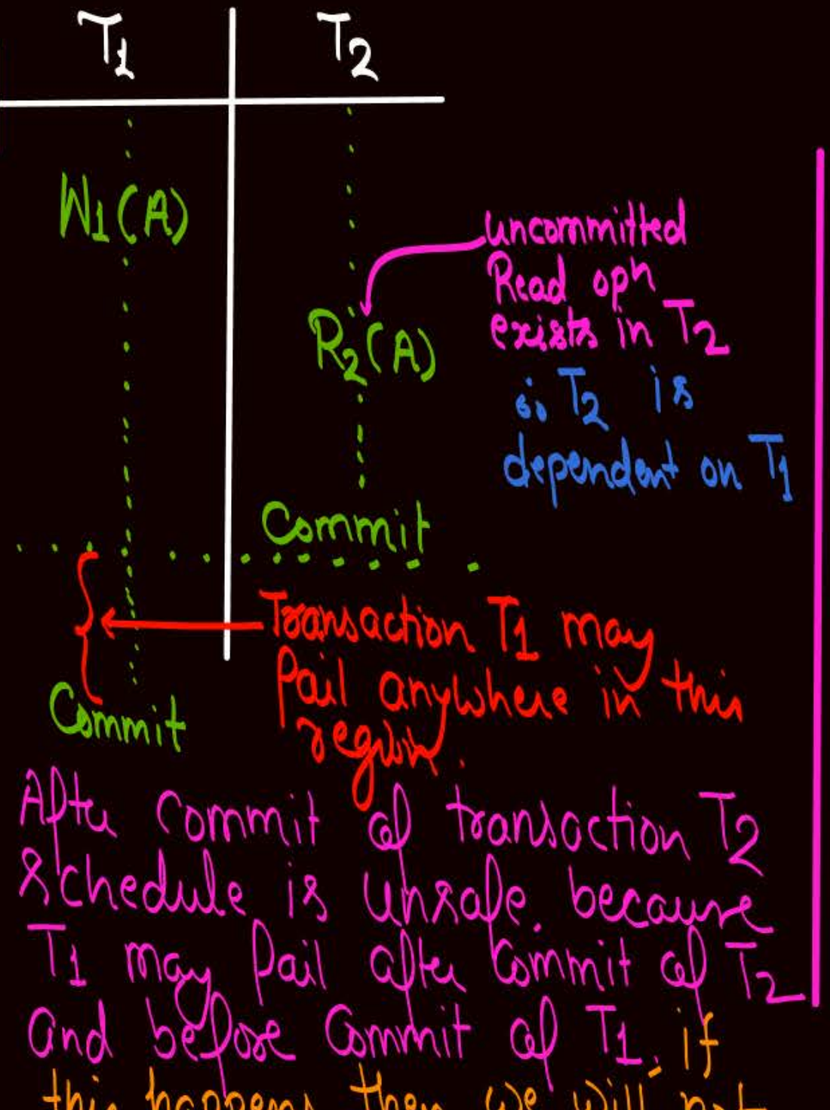


- No transaction has performed its Commit oph
  ie If any failure occur, then we can rollback both transactions
  i.e, we will be able to recover from failure
  Hence, Schedule is recoverable schedule

Irrecoverable Schedule

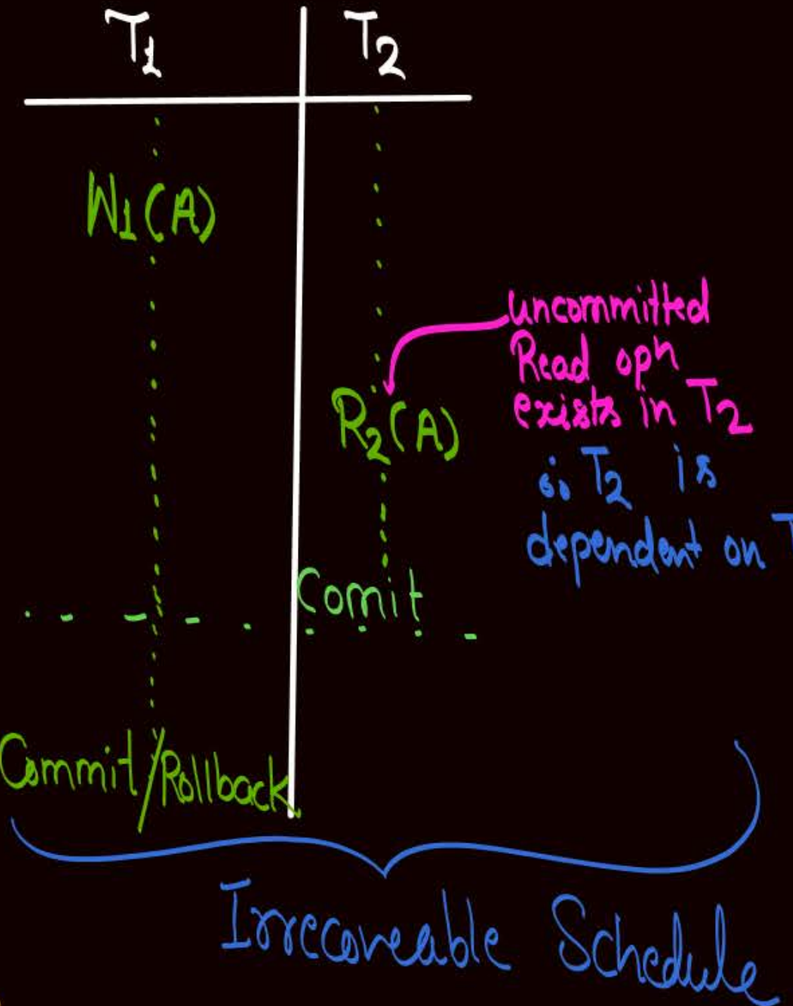


Irrecoverable Schedule



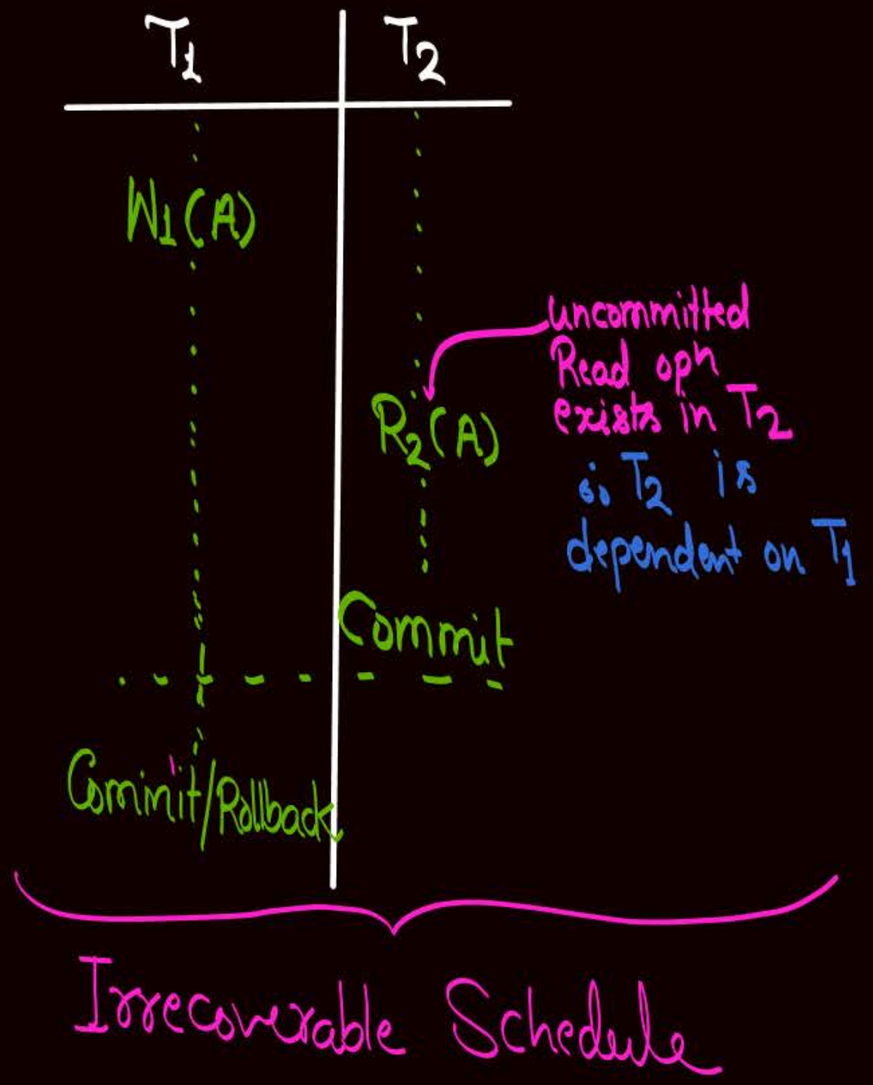
After commit of transaction T_2 schedule is unsafe because T_1 may fail after commit of T_2 and before Commit of T_1, if this happens then we will not be able to recover from failure.

Hence it is also an irrecoverable schedule



Irrecoverable Schedule

Irrecoverable Schedule



* Let two transactions T_1 & T_2 exists in a schedule such that T_2 is dependent on T_1,

  If transaction T_2 {i.e. dependent transaction} Commit before the commit/Rollback of transaction T_1 {ie, transaction on which T_2 depends}, then such schedule is called "Irrecoverable Schedule"

Irrecoverable Schedule

Uncommitted read oph are necessary (but not sufficient) for a schedule to be irrecoverable Schedule.

i.e, ① If uncommitted read oph does not exist in a schedule then that schedule is always recoverable

and ② If uncommitted read oph exists in a schedule then that schedule may or may not be an irrecoverable schedule.

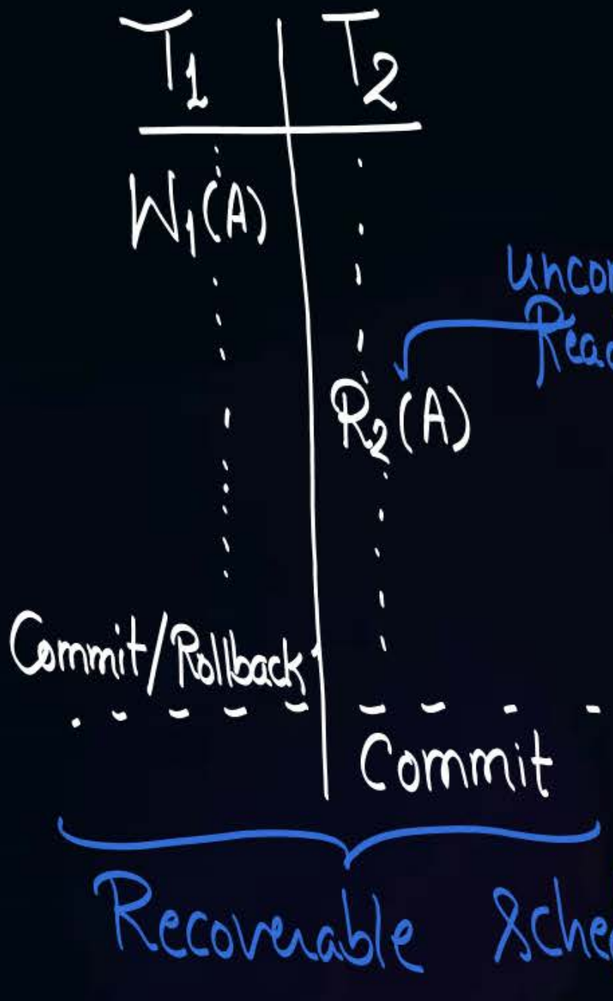


Topic : Recoverable schedule

For a schedule to be called recoverable schedule,

either, uncommitted read oph must not exist in the schedule

or, if uncommitted read oph exists in the schedule, then Commit oph of dependent transaction must appear after the Commit/Rollback of the transaction on which it depends.



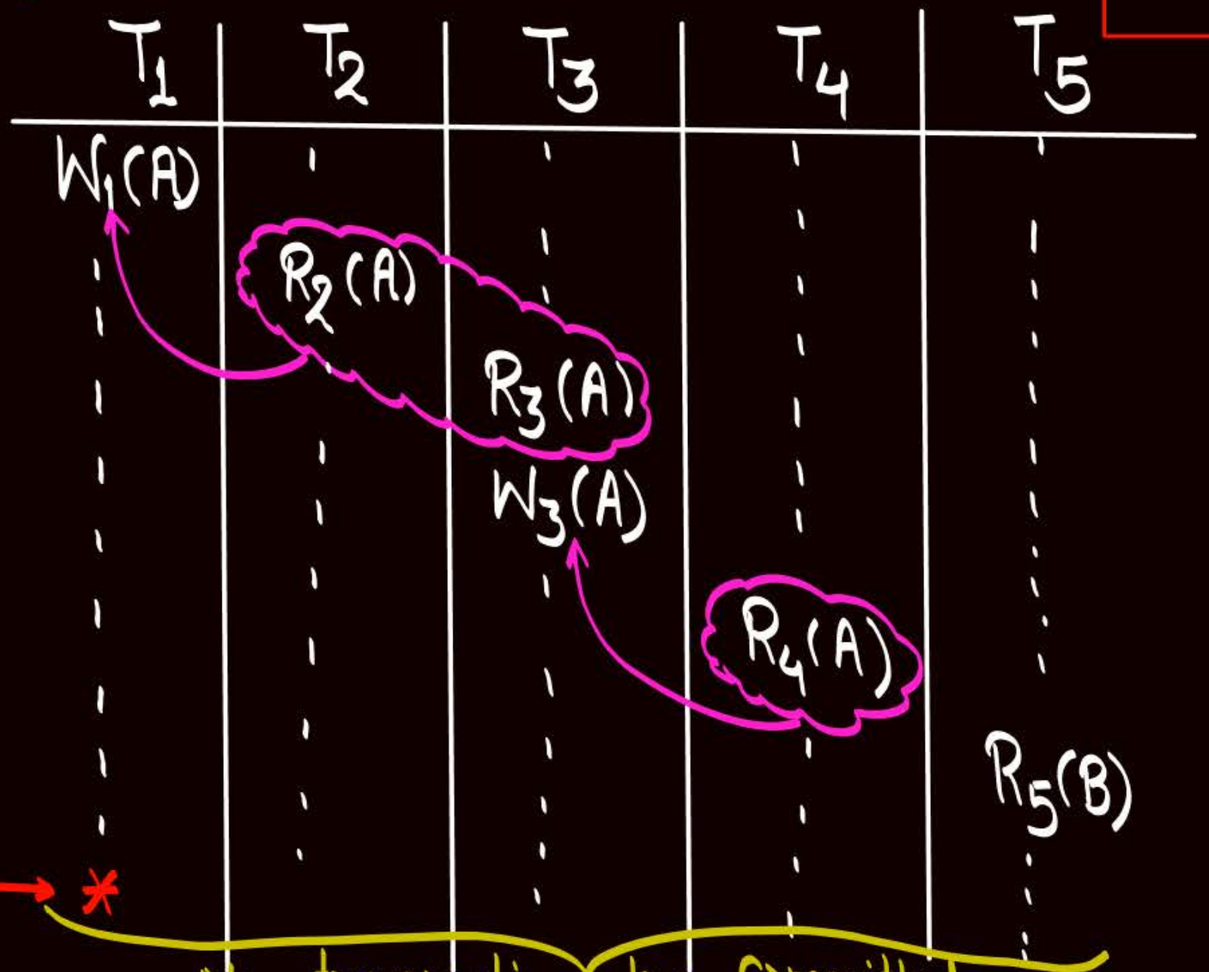


Topic : Cascading rollback problem

If failure of one transaction results in rollback of a set of other transactions along with failed transaction, then it is called Cascading rollback.

And we say that the Schedule suffer from Cascading rollback problem.

Cascading rollback problem



Cascading rollback increases Wastage of CPU time & IO time

- If T_1 fails, we rollback T_1.
- Because of rollback of T_1 we need to rollback transactions dependent on T_1, i.e, T_2 & T_3
- No other transaction depends on T_2 but T_4 depends on T_3.
- ∴ Because of rollback of T_3 we will rollback T_4 as well.
- No other transaction depends on T_4

Because of failure of T_1, we need to rollback T_2, T_3 & T_4 as well along with T_1 ∴ Cascading rollback Problem exist.



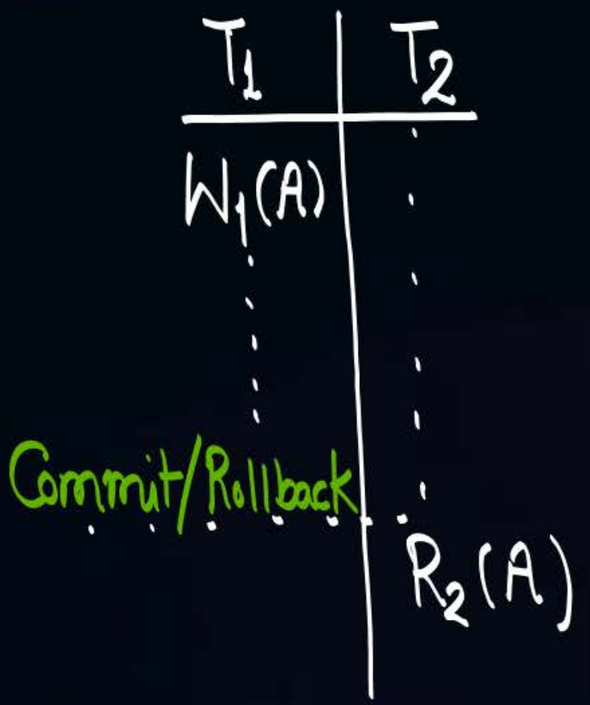
Topic : Cascadeless recoverable schedule

- If uncommitted read opn exist in the schedule then dependency will exist among the transactions, and if dependency exists among the transaction then because of failure of one transaction we may have to rollback some other transactions as well, i.e. Cascading rollback Problem will exist.

- For Cascadeless recoverable schedule, uncommitted read operation must not be allowed



Topic : Cascadeless recoverable schedule

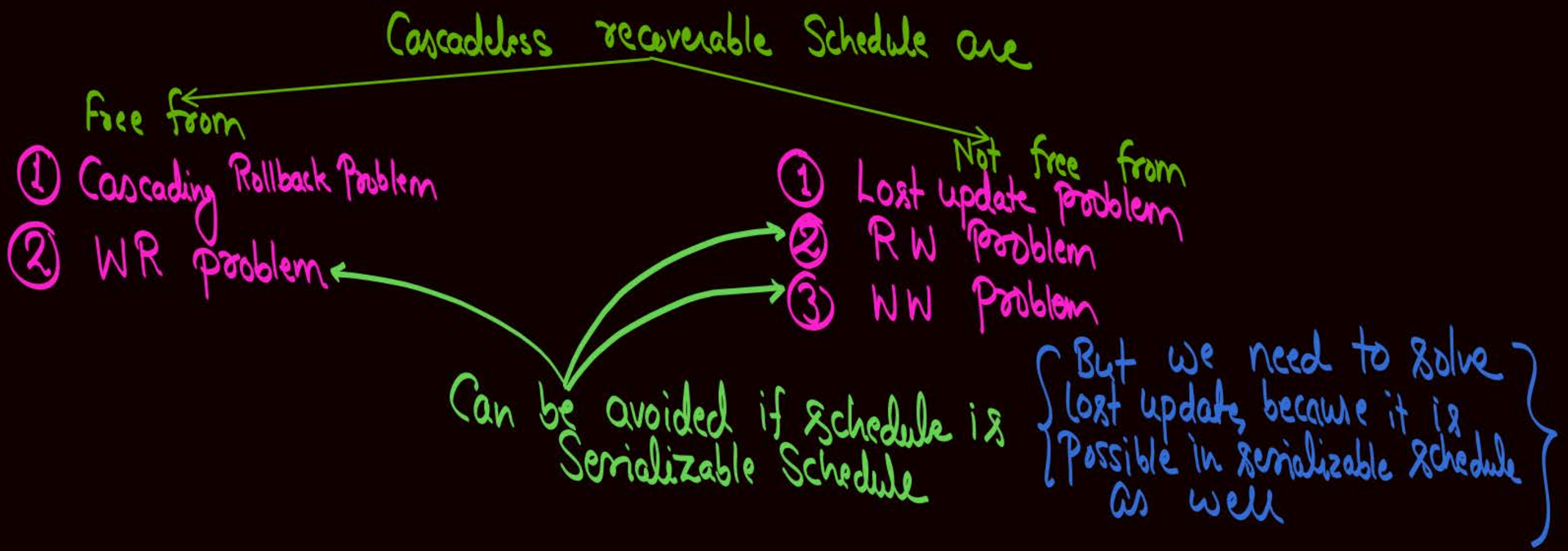


- For cascadeless recoverable schedule uncommitted read op^n must not be allowed.
- For cascadeless recoverable schedule,
  If transaction T_1 updates the dataitem(A), then no other transaction should be allowed to read the value of dataitem 'A' written by T_1 until the commit/Rollback of T_1.

Note:- Uncommitted read oph are possible in Serializable as well as non-Serializable Schedule.

i.e., Cascading rollback problem is possible in Serializable Schedule as well.

Hence we had to find the solution for Cascading rollback problem.





Topic : Strict recoverable schedule

Strict recoverable schedule is Cascadeless recoverable schedule as well as it is free from lost-update problem

(Cascadeless Recoverable Schedule (i.e, No uncommitted Read) + No lost-update Problem (i.e, No simultaneous write-write opn)) = Strict Recoverable Schedule

T_1	T_2
$W_1(A)$	
Commit/Rollback	
	$R_2(A)$

+

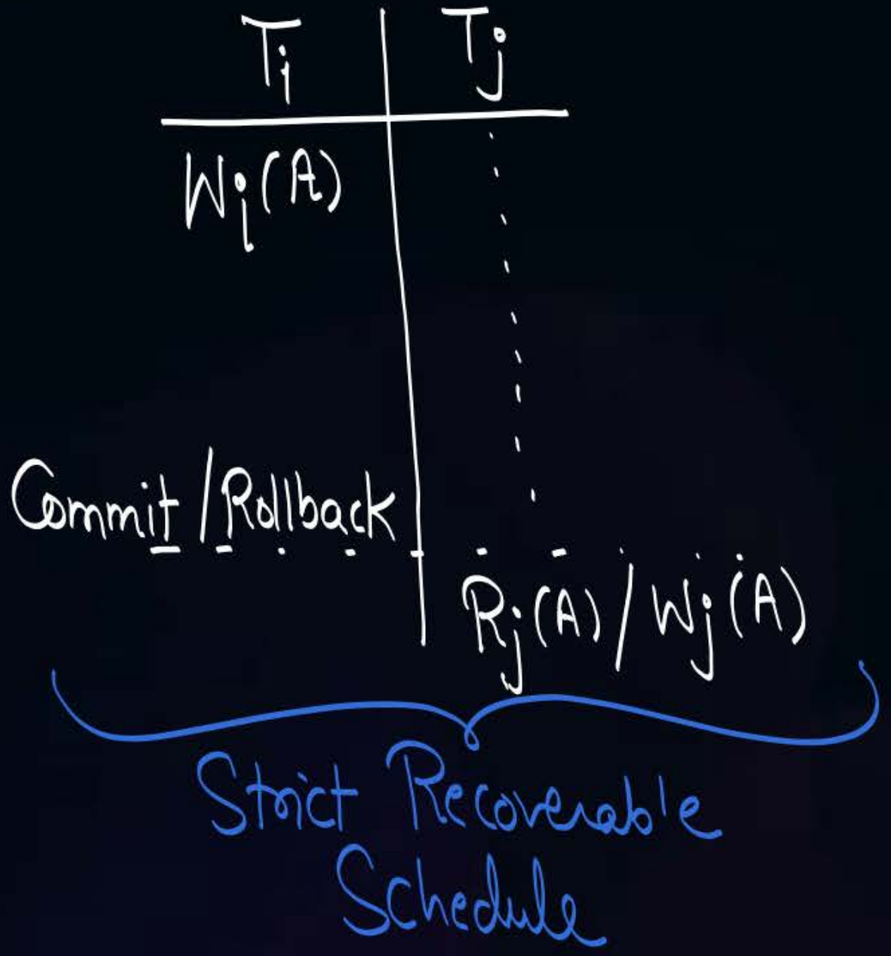
T_1	T_2
$W_1(A)$	
Commit/Rollback	
	$W_2(A)$

=

T_1	T_2
$W_1(A)$	
Commit/Rollback	
	$R_2(A)/W_2(A)$



Topic : Strict recoverable schedule



For a schedule to be called strict recoverable, if transaction Ti updates any dataitem 'A', then no other transaction Tj is allowed to read or write the dataitem 'A' until the commit or Rollback of transaction Ti

Strict Recoverable Schedule

Ans.

Free from

1. Cascading Rollback Problem ✓
2. Lost-update problem ✓
3. WR problem
4. WW Problem

Not free from

1. RW problem

Can be solved by Ensuring "Serializability"

Criteria for Consistency :-

1. Schedule must be Strict recoverable
2. Schedule must be Serializable

These two can solve all problems



Topic : Classification of schedule

Classification based on Recoverability

1. Irrecoverable schedule
2. Recoverable schedule
3. Cascadeless rollback recoverable schedule
4. Strict recoverable schedule

Classification based on Serializability

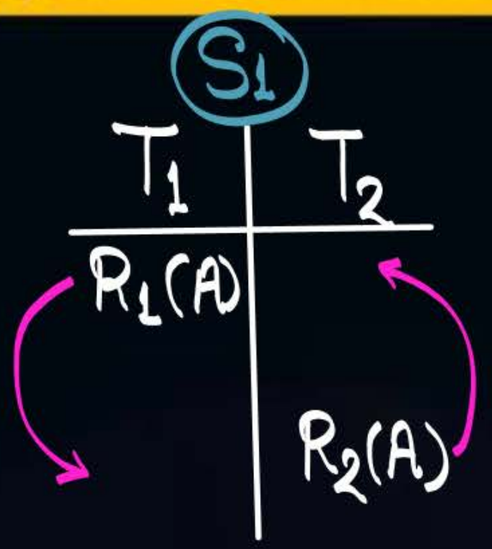
1. Conflict serializable schedule
2. View serializable schedule

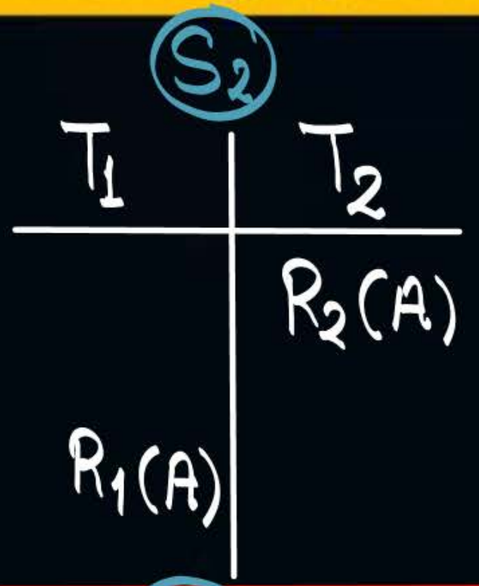
Conflict Serializable Schedule

Topic : Conflicting and non-conflicting operations



1

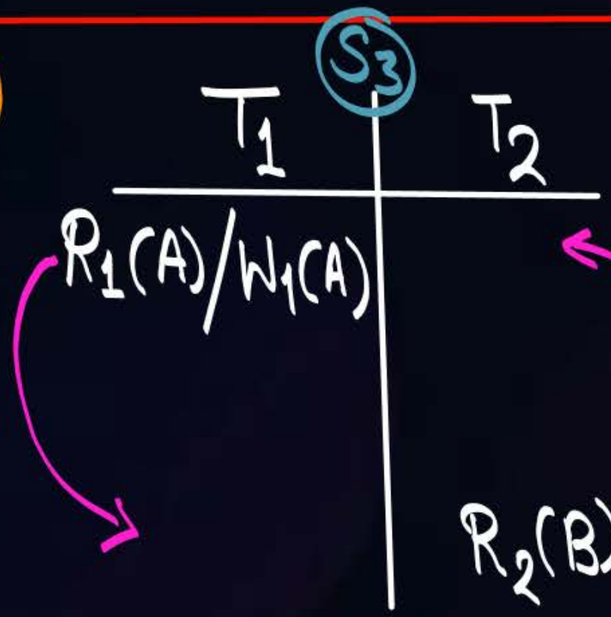


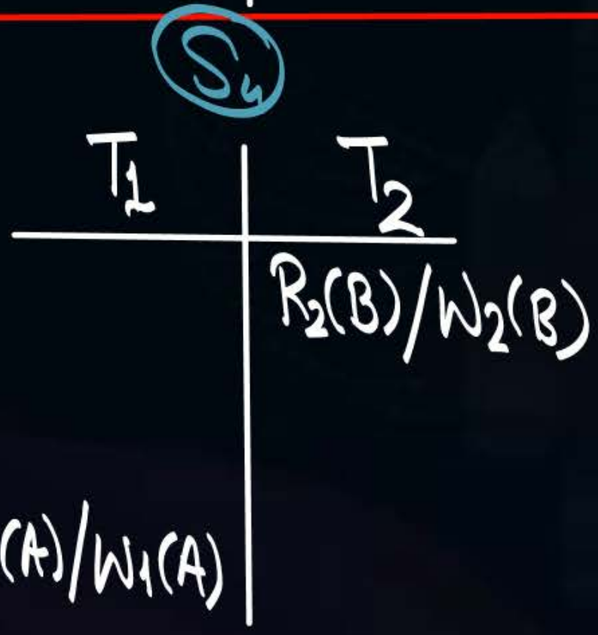


$S_1 \equiv S_2$

∴ $R_i(A)$ & $R_j(A)$ are non-conflicting operation

2



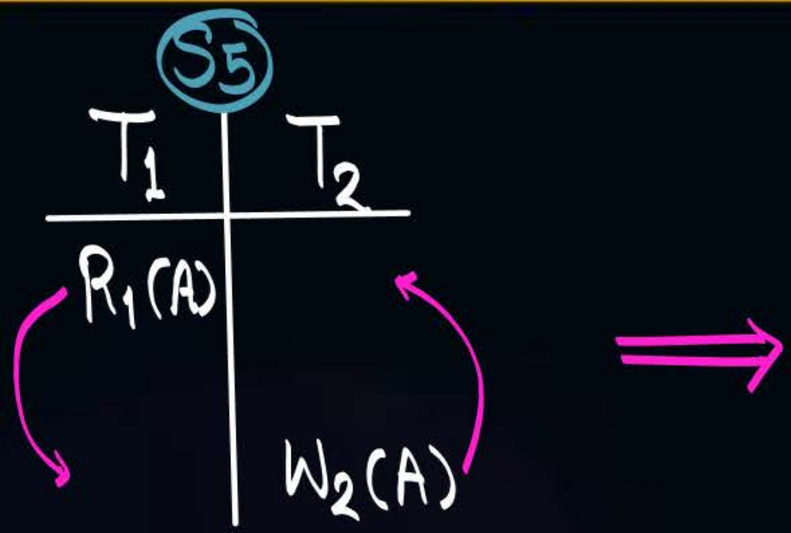


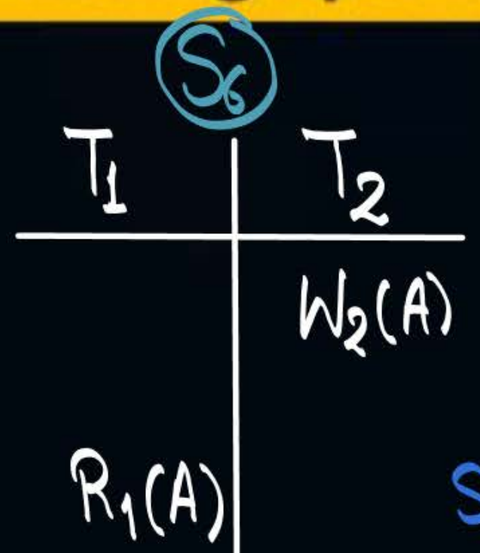
$S_3 \equiv S_4$

∴ $R_i(A)/W_i(A)$ are non-conflicting with $R_j(B)/W_j(B)$

Topic : Conflicting and non-conflicting operations

③



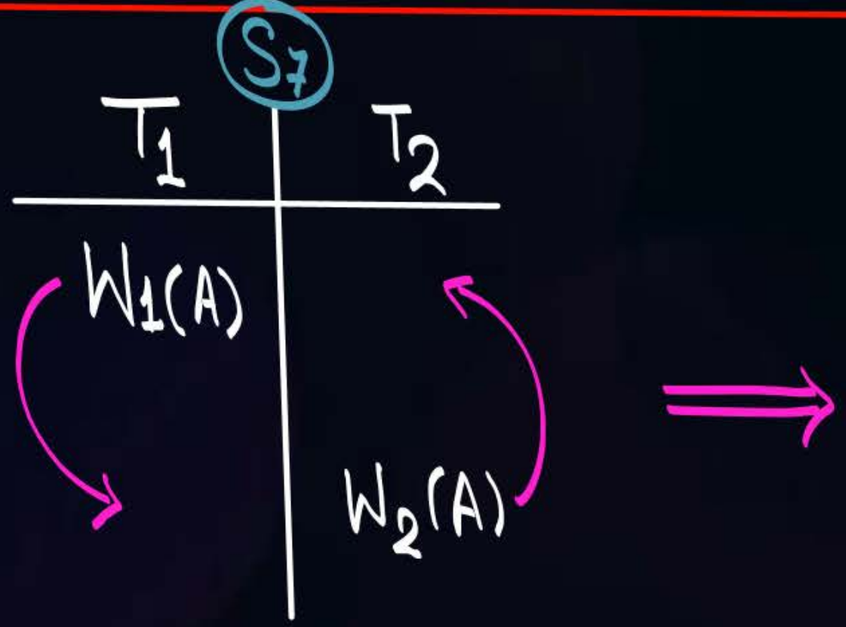


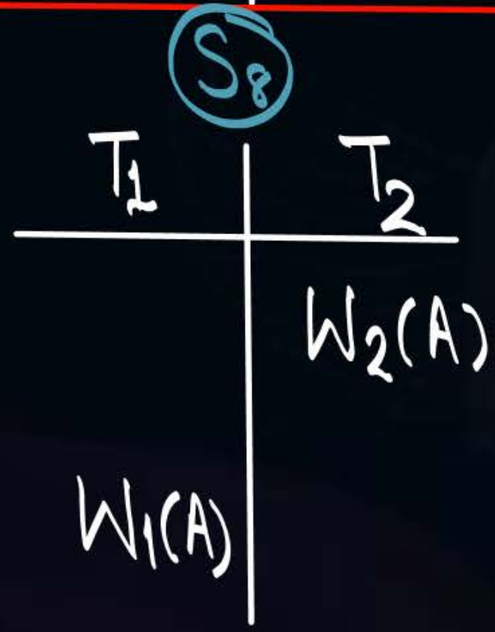
$S_5 \not\equiv S_6$

∴ $R_i(A)$ & $W_j(A)$ are Conflicting operations

Similarly, $W_i(A)$ & $R_j(A)$ are also conflicting oph

④





$S_7 \not\equiv S_8$

∴ $W_i(A)$ & $W_j(A)$ are Conflicting oph



Topic : Conflicting and non-conflicting operations

- Conflicting Operations : Two operations are called as conflicting operations if all the following conditions hold true for them -
  - Both the operations belong to different transactions
  - Both the operations are on the same data item
  - At least one of the two operations is a write operation

We can never swap the position of two operations within the same transaction



Topic : Conflicting and non-conflicting operations

- **Non-Conflicting Operations :** Two operations are non-conflicting if and operations only if
  - Both the operations are on different data items

    or
  - Both the operations are read operations

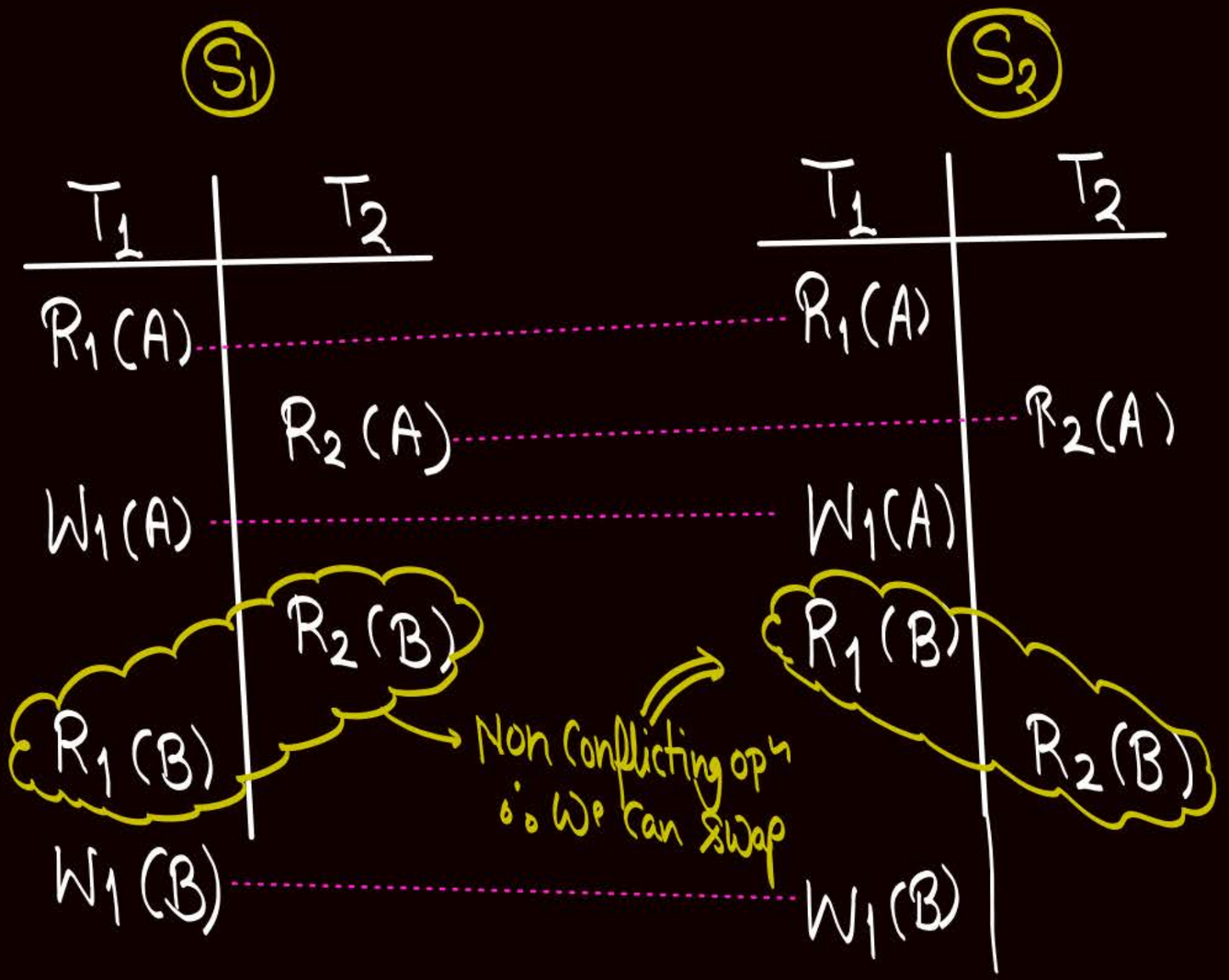


Topic : Conflict equivalent schedule

Consider two schedules Si and Sj,

If one can be transformed into another by swapping the position of some of the non-conflicting operations in it, then schedule Si and Sj are called conflict equivalent schedule.

eg: Consider two Schedules S_1 & S_2



- S_1 can be converted into S_2, by swapping the position of some of the non-conflicting operations.

∴ S_1 is Conflict equivalent to S_2

eg: Consider two Schedules S_1 & S_2

S_1

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
$R_1(B)$	
$W_1(B)$	
	$R_2(B)$

↳ Conflicting opn ∴ We can not move $R_2(B)$ of T_2 above $W_1(B)$ of T_1

S_2

T_1	T_2
$R_1(A)$	
$W_1(A)$	
	$R_2(A)$
	$R_2(B)$
$R_1(B)$	
$W_1(B)$	

- S_1 can not be Converted into S_2, by swapping the position of some of the non-conflicting operations.

∴ S_1 & S_2 are not Conflict equivalent

⇒ Hence S_1 can not be Converted into S_2

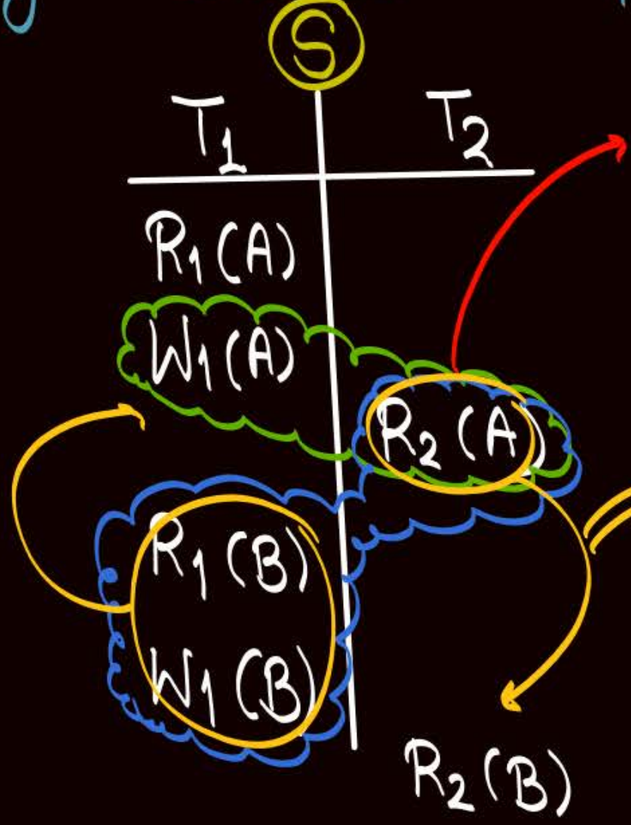


Topic : Conflict serializable schedule

- If a given schedule can be converted into a serial schedule by swapping the positions of some of its non-conflicting operations, then it is called a conflict serializable schedule.

* If given schedule is conflict equivalent to at least one of the serial schedule, then given schedule is conflict serializable schedule.

eg: Consider the following schedule 'S'



Check, whether the schedule 'S' is Conflict Serializable schedule or not?

Conflicting opn

∴ $R_2(A)$ can not be moved above $W_1(A)$

Hence 'S' can not be conflict equivalent to serial schedule $T_2 \rightarrow T_1$

$R_2(A)$ is non-conflicting with $R_1(B)$ as well as $W_1(B)$

∴ We can swap the position and we can bring $R_2(A)$ below $R_1(B)$ & $W_1(B)$

Hence 'S' can be converted in to serial schedule $T_1 \rightarrow T_2$

Schedule S is Conflict Equivalent to at least one serial schedule

∴ 'S' is Conflict Serializable schedule

Two serial schedules are possible over the transactions of schedule S.

① $T_1 \rightarrow T_2$

T_1	T_2
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	
	$R_2(A)$
	$R_2(B)$

② $T_2 \rightarrow T_1$

T_1	T_2
	$R_2(A)$
	$R_2(B)$
$R_1(A)$	
$W_1(A)$	
$R_1(B)$	
$W_1(B)$	

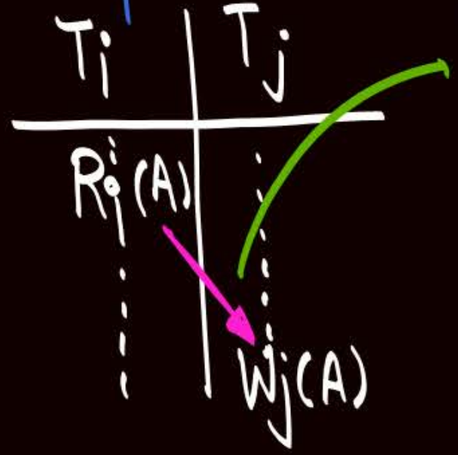
and Conflict equivalent serial schedule is $T_1 \rightarrow T_2$

Note

① If schedule S is a Conflict Serializable schedule, then it is guaranteed that schedule S is a Serializable schedule

② If schedule S is not a Conflict Serializable schedule, then schedule S may or may not be a Serializable schedule

③ Conflicting pairs of operations will define the precedence order in which transactions must execute in an equivalent serial schedule



Precedence defined by this Conflicting pair of opⁿ is Tj then Tj

∴ in any Equivalent Serial schedule transaction Ti must execute before Tj



Topic : Precedence graph

Set of vertices
Set of edges

Precedence graph $G = (V, E)$ is a directed graph.

Where Set of vertices = V = Set of all transactions of the schedule

Set of Edges = E = Set of directed edges based on Precedence order defined by pairs of Conflicting operations

S_1

T_1	T_2
$R_1(A)$	
	$W_2(A)$



Precedence graph w.r.t. schedule S_1

S_2

T_1	T_2
$R_1(A)$	
	$W_2(A)$
$W_1(A)$	



Precedence graph w.r.t. schedule S_2

S_3

T_1	T_2
$R_1(A)$	
	$W_2(B)$
$R_1(A)$	



No pair of Conflicting op^n in schedule ∴ No edge in Graph

Precedence graph w.r.t. schedule S_3



Topic : Testing condition for conflict serializable schedule

① If precedence graph is acyclic, then given schedule is Conflict Serializable schedule (hence serializable), and Conflict equivalent serial schedules can be given by "Topological order" of precedence graph.

② If precedence graph is Cyclic, then given schedule is not a Conflict serializable schedule, (it may or may not be serializable)



Topic : Topological order

In-degree of a vertex 'v' is defined by number of edges incoming to vertex 'v'.

Let $G_T = (V, E)$ be a precedence graph

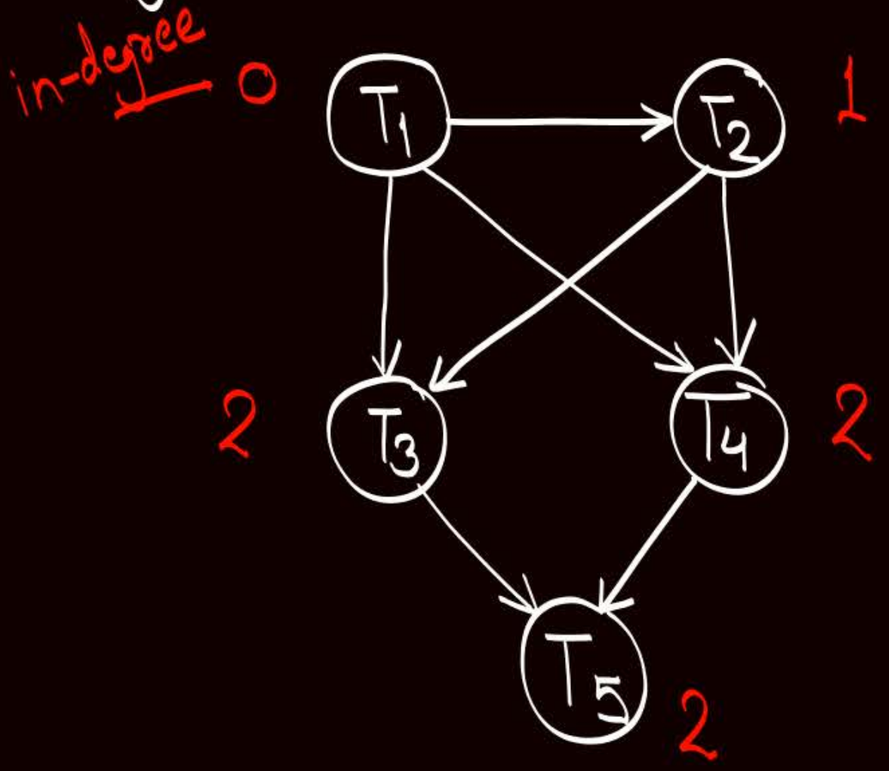
Step-1:- Visit a vertex $V \in G_T$, such that in-degree of vertex $V = 0$. and Visiting the vertex 'v' delete vertex 'v' from graph G_T.
{when we delete a vertex, associated edges will also be deleted}

Step-2:- Repeat "step-1" until graph becomes Empty {i.e. When there is no vertex in the graph}

The order in which vertices of precedence graph G_T can be visited is called topological ordering w.r.t. given Precedence graph

There may be more than one topological ordering for the given precedence graph

eg: Consider the following precedence graph w.r.t. some schedule "S"



Identify all topological ordering w.r.t. given Precedence graph

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

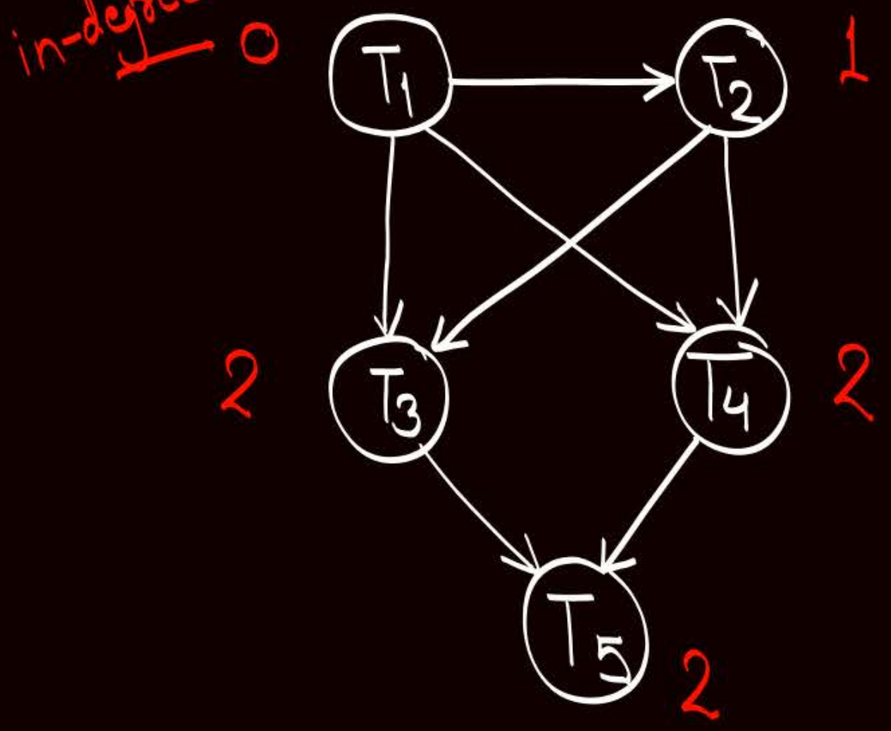
There are two topological ordering for given precedence graph.

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

&

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

eg: Consider the following precedence graph w.r.t. some schedule "S"



Find all the serial schedules that are conflict equivalent to given schedule 'S'

There are two topological ordering for given precedence graph.

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

&

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

∴ Given schedule will be conflict equivalent to two serial schedules, and those serial schedules are

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5$

&

$T_1 \rightarrow T_2 \rightarrow T_4 \rightarrow T_3 \rightarrow T_5$

} i.e, same as topological ordering



2 mins Summary

- Topic: Classification based on serializability
- Topic: Conflicting and non-conflicting operations
- Topic: Conflict serializable schedule
- Topic: Precedence graph
- Topic: Topological order

Slide

THANK - YOU